OpenSAI: A Distributed Cooperative Computing Platform for Supporting Edge Intelligence

Kun Yang[1,2,†], Yuntian Shi[1,†], Yu Du[1], Jingnan Cai[1], Bobo Ju[3], Jing Wang[2]
Zhaohui Yang[2], Zhenchao Ma[4], Liang Song[1,*]
[1] Fudan University, [2] Zhejiang University, [3] Shanghai Shentong Metro Group Co., Ltd., [4] The University of British Columbia

***Abstract*—Cooperative edge computing has emerged as a key paradigm for deploying distributed intelligence in dynamic environments, particularly for latency-sensitive multimedia applications such as real-time video analytics, augmented reality, and multi-view perception. However, existing systems grapple with several challenges: fluctuating network topologies, meeting task-specific Quality of Service (QoS) demands under resource constraints, and the heterogeneity of edge nodes. To address these issues, we introduce OpenSAI, a distributed cooperative computing platform that optimizes task execution via task decomposition, resource coordination, and dynamic configuration. OpenSAI adopts a three-layer architecture, with its core layer integrating three synergistic components. Specifically, the Computation Distribution Component partitions computational tasks into parallel subtasks based on operation-level dependencies, while the Cluster Management Component maintains a stable cooperative environment by monitoring node status and allocating subtasks via a greedy scheduling algorithm. Concurrently, the Adaptation Optimization Component enhances execution efficiency by adjusting task parameters in response to real-time resource fluctuations, thereby ensuring QoS compliance while preventing overload on resource-constrained nodes. Experimental results demonstrate that OpenSAI significantly reduces task latency while ensuring high reliability in heterogeneous and dynamic edge environments. Compared to state-of-the-art methods, OpenSAI exhibits a superior balance between efficiency, adaptability, and system stability.**

***Index Terms*—Distributed Machine Learning, Task Scheduling, Cooperative Intelligence, Mobile Systems, Multimedia Processing**

I. Introduction

Edge computing has emerged as a key distributed paradigm that brings computational resources closer to data sources, thereby enabling latency-sensitive and data-driven applications within the Internet of Things (IoT) ecosystem [1], [2]. By reducing data transmission distance, edge computing facilitates real-time analytics and decision-making for intelligent multimedia services, including industrial automation, augmented reality [3], [4], and AI-assisted healthcare. Moreover, the proliferation of Deep Learning drives the deployment of lightweight neural networks on edge devices to support mission-critical multimedia tasks such as video surveillance analytics [5] and autonomous vehicle [6]. However, the inherent resource constraints of edge devices necessitate strategic optimizations to balance efficiency with service quality, such as deploying compressed DNN models or dynamically offloading computations [5]. Consequently, the paradigm shift toward **edge intelligence** represents a fundamental transformation in how distributed systems process and analyze real-time IoT data at the network periphery.

However, traditional edge computing architectures often contend with resource fragmentation and geographical constraints, limiting their efficacy for complex intelligent tasks. To address these challenges, recent advances have given rise to cooperative edge systems, where interconnected nodes collaboratively execute workloads via peer-to-peer coordination [7]. By leveraging localized communication and decentralized resource pooling, cooperative edge systems enable nodes to surpass their individual capacities and collectively support computation-intensive applications [8]. Compared to centralized edge-cloud models, cooperative edge computing offers superior scalability and fault tolerance by eliminating single points of failure and accommodating large-scale deployments more effectively [9]. For instance, distributed video analytics systems can achieve real-time object recognition across smart cities by dynamically partitioning detection tasks among neighboring edge devices [10], [11]. In addition, cooperative frameworks inherently support multi-tenancy, allowing diverse application providers to share underlying edge resources through coordinated task scheduling.

Despite its potential, the practical deployment of distributed cooperative computing is impeded by three critical challenges. First, inherent heterogeneity in node capabilities complicates resource utilization, as conventional partitioning strategies struggle to adapt to diverse hardware configurations, leading to suboptimal load balancing. For instance, systems integrating high-end GPUs with legacy IoT devices require adaptive decomposition to fully leverage disparate resources [12], [13]. Second, the volatility of edge environments leads to topological instability, characterized by high node churn due to mobility or power constraints, and static cluster management approaches fail to ensure service continuity in such dynamic settings, exemplified by vehicular networks where nodes form transient alliances [14]. Third, the tension between stringent QoS requirements and limited resources necessitates intelligent parameter adaptation, yet fixed configurations often lead

†Equal contribution. *Corresponding authors. This work was supported by National Key Research and Development Program of China, Project No.2024YFE0200700, Subject No.2024YFE0200703.

to resource overcommitment because existing systems typically lack mechanisms to adjust task parameters in response to real-time resource status [8], [15]. Therefore, addressing these challenges requires a holistic approach that integrates task orchestration, cluster resilience, and adaptive QoS management.

To address these challenges, we propose OpenSAI, a distributed cooperative computing platform designed to enable efficient collaboration among heterogeneous nodes in dynamic environments. OpenSAI employs a three-layer architecture that decouples application logic from task management and network services. At its core, the Generalized Smart Service layer integrates three synergistic components to optimize distributed execution. Specifically, the Computation Distribution Component analyzes tasks' operation-level dependency graph to partition it into parallel subtasks, enabling fine-grained distributed execution. Meanwhile, the Cluster Management Component ensures a stable cooperative environment by continuously monitoring node status and resources, dynamically managing cluster membership, and allocating subtasks via a greedy scheduling algorithm that balances workloads. Additionally, the Adaptation Optimization Component enhances efficiency by adjusting task parameters based on real-time resource information, thus achieving bidirectional adaptation between application requirements and system constraints. Collectively, these components ensure that task execution meets QoS requirements without overloading resource-constrained nodes. We implement and evaluate OpenSAI on two representative workloads, including π computation and multi-agent cooperative perception, under various dynamic edge computing scenarios. Experimental results demonstrate that OpenSAI significantly reduces task completion time and improves success rates compared to existing approaches, showcasing its superior adaptability and efficiency in unstable environments. The contributions are summarized as follows:

- We propose OpenSAI, a distributed cooperative computing platform that enables efficient and adaptive task execution for edge intelligence. OpenSAI achieves this through a synergistic architecture that unifies task decomposition, dynamic cluster management, and adaptive parameter tuning.
- We design a Computation Distribution Component that decomposes complex tasks into parallel subtasks with operational dependencies. Also, our Cluster Management and Adaptation Optimization components enable dynamic cluster coordination and real-time parameter tuning to ensure efficient and stable execution.
- We conduct a comprehensive evaluation of OpenSAI on representative workloads in dynamic edge environments. Our results demonstrate that the platform outperforms state-of-the-art approaches in both task latency and reliability, proving its superior efficiency and resilience.

II. Related Work

Task Offloading for Edge Intelligence. Task offloading addresses the conflict between intensive applications and resource-constrained edge devices. Foundational works established partitioning mechanisms, such as Neurosurgeon [16], which pioneered layer-level DNN partitioning, and EdgeML [17], which employed Reinforcement Learning (RL) for dynamic offloading. However, these methods primarily focus on single client-server models. More recent advances have shifted to cooperative settings, often using Multi-Agent DRL (MADRL) to coordinate nodes [18]. While these approaches advance cooperative execution, they often lack true operation-level granularity for Directed Acyclic Graph (DAG) tasks and flexible strategies to balance conflicting objectives like parallelism versus communication overhead [19]. In contrast, our work, through OpenSAI's Computation Distribution Component, introduces a general-purpose mechanism for adaptive, fine-grained DAG partitioning. It provides diverse strategies to dynamically balance parallelism against resource and communication costs, directly addressing this critical gap.

Joint Resource Allocation for Edge Computing. Early approaches often managed computational [20] and communication [21] resources in isolation, leading to suboptimal performance. While subsequent research shifted to joint allocation using techniques like Lyapunov optimization [22], these methods frequently overlook heterogeneous QoS needs and struggle with environmental dynamism and the overhead of global state collection [19]. Accordingly, OpenSAI's Cluster Management Component employs a hybrid optimization algorithm, integrating a greedy strategy with node matching. It allocates tasks based on comprehensive node states to manage heterogeneous resources and avoids local optima without requiring expensive global state synchronization.

III. Preliminaries

In cooperative edge computing system, a complex computational task T is partitioned into a set of K subtasks, denoted as $\{T_j\}_{j=1}^K$. Each subtask T_j is characterized by an operation f_j and input data $\mathcal{I}_j$, and is assigned to one or more edge nodes for execution. Accordingly, the output of each subtask is a partial result $y_j = f_j(\mathcal{I}_j)$, and the final result of the original task T is obtained by aggregating all partial results using a function $\mathcal{A}$:

$$Y = \mathcal{A}(\{y_j\}_{j=1}^K). \tag{1}$$

We formalize two representative distributed tasks below.

π Computation. The task of estimating π via the Monte Carlo method with a total of S_{total} samples is partitioned into K independent subtasks $\{T_j\}_{j=1}^K$. Each subtask T_j is assigned to process a subset of S_j samples, where $\sum_{j=1}^K S_j = S_{total}$. The subtask's operation f_j is to count the number of its samples, S_j^{ins}, that fall within a unit circle. Its partial result is the tuple $y_j = (S_j^{ins}, S_j)$. The final estimation of π is obtained by aggregating all partial results, where the aggregation function $\mathcal{A}$ is defined as:

$$Y = \mathcal{A}(\{y_j\}_{j=1}^K) = 4 \times \frac{\sum_{j=1}^K S_j^{ins}}{\sum_{j=1}^K S_j}. \tag{2}$$

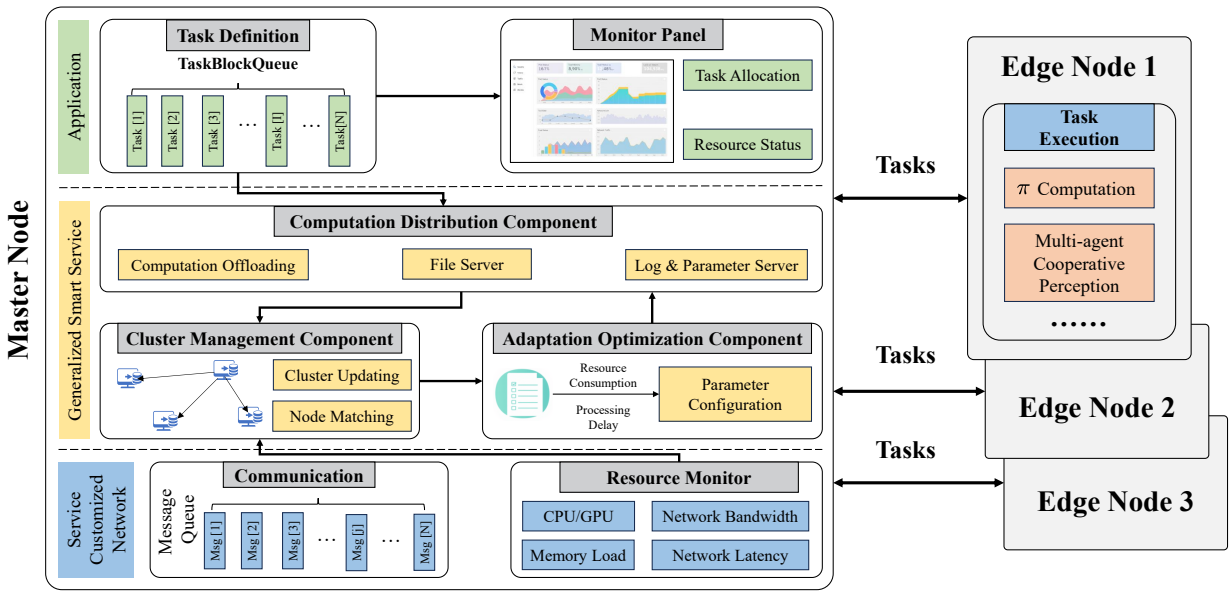


Fig. 1: Overview of the OpenSAI architecture. It illustrates the workflow from task submission to execution. The Generalized Smart Service layer leverages the Computation Distribution, Cluster Management, and Adaptation Optimization components to dynamically partition tasks and configure parameters. Subtasks are then dispatched to edge nodes for local processing and result aggregation.

Multi-agent Cooperative Perception. In multi-agent cooperative perception, perception results from K collaborating edge nodes are fused to obtain a comprehensive global view. Each node $j \in \{1, \ldots, K\}$ is responsible for a local object detection subtask T_j, which involves processing local sensory data $\mathcal{I}_j$ with a local model Φ_j. The partial result y_j is a set of local bounding box detections:

$$y_j = \Phi_j(\mathcal{I}_j). \tag{3}$$

A designated central node aggregates the set of partial results $\{y_j\}_{j=1}^{K}$. The aggregation function $\mathcal{A}$ is typically Non-Maximum Suppression operation, which filters redundant detections to produce the final global perception result Y:

$$Y = \text{NMS}(\{y_j\}_{j=1}^{K}). \tag{4}$$

IV. Methodology

In this section, we present OpenSAI, a general-purpose distributed platform designed to orchestrate computing, communication, and storage resources across dynamic edge-cloud environments. By facilitating bidirectional adaptation between system constraints and AI application requirements, OpenSAI ensures execution efficiency and service reliability.

A. Overview of OpenSAI

As illustrated in Figure 1, OpenSAI adopts a three-layer architecture: the *Application Layer*, *Generalized Smart Service*, and *Service Customized Network*. While the Application Layer is responsible for task definition, submission, and execution monitoring, the core Generalized Smart Service layer integrates three synergistic components: 1) The **Computation Distribution Component** analyzes operation-level dependencies within tasks to enable parallel execution; 2) The **Cluster Management Component** maintains the cooperative node cluster and schedules subtasks based on real-time resource monitoring; 3) The **Adaptation Optimization Component** dynamically configures task parameters to align application utility with system constraints. The workflow proceeds sequentially: upon task submission, OpenSAI performs task partitioning, node matching, and parameter optimization. The final configuration is distributed to cooperative nodes for code synchronization and subtask execution, with results finally aggregated by the Computation Distribution Component.

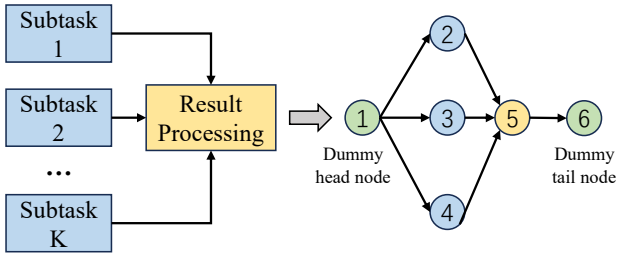


Fig. 2: Schematic diagram of a computing task comprising K subtasks. The computation offloading module transforms the task into a DAG by modeling inter-subtask dependencies.

B. Computation Distribution Component

To efficiently leverage distributed edge resources, the Computation Distribution Component manages task partitioning and code synchronization. It comprises a File Server for maintaining a consistent execution environment and a Computation Offloading module for logic partitioning.

We model a computing task T (e.g., multi-agent cooperative perception) as a Directed Acyclic Graph (DAG) $G = (V, E)$,

where $V = \{v_1, v_2, ..., v_K\}$ represents the set of K subtasks, and E denotes data dependencies. To complete the graph topology, dummy nodes v_{head} and v_{tail} are added. For any subtask $v_i \in V$, its predecessor set is $\text{Pred}(v_i) = \{u \in V \mid (u, v_i) \in E\}$. Subtask v_i becomes executable only when:

$$\forall u \in \text{Pred}(v_i), \text{State}(u) = \text{Completed}. \tag{5}$$

Based on the DAG topology and real-time states (e.g., resource consumption, latency), the module dynamically divides V into m execution partitions $\mathcal{P} = \{P_1, P_2, ..., P_m\}$, where $\bigcup_{j=1}^{m} P_j \subseteq V$. Two strategies are employed:

- **Full Parallelism Strategy**: $P_j = \{v_j\}$. Each parallelizable subtask is assigned to an independent node (see Figure 2) to minimize completion time.
- **Grouped Strategy**: $P_j = \{v_a, v_b, ...\}$. Multiple subtasks are aggregated onto a single node based on dependencies, reducing the number of required nodes while preserving parallelism.

C. Cluster Management Component

This component ensures cluster stability and optimizes subtask allocation. To address the volatility of mobile edge topologies, we employ a dynamic maintenance mechanism. Let $\mathcal{D}_t$ denote the set of active nodes at time t. The cluster manager broadcasts heartbeats at an interval T_{heart}. If a node d_i fails to respond for N_{heart} consecutive cycles, the available set is updated:

$$\mathcal{D}_{t+1} = \mathcal{D}_t \setminus \{d_i \mid \text{Count}_{fail}(d_i) \geq N_{heart}\} \cup \mathcal{D}_{new}, \tag{6}$$

where $\mathcal{D}_{new}$ represents the set of newly discovered nodes.

Given a valid resource pool, the core challenge is mapping a subtask to the optimal node $d^* \in \mathcal{D}_t$. To avoid confusion with the sample size S defined in preliminaries, we define the **utility score** $U(d_i, t)$ for node d_i, which aggregates normalized network metrics (R_{net}), computational capabilities (R_{comp}), and historical stability (H_{stab}):

$$U(d_i, t) = \cdot R_{net}(d_i) + \cdot R_{comp}(d_i) + \lambda \cdot H_{stab}(d_i), \tag{7}$$

where λ is weighting coefficient. Note that R_{net} and R_{comp} represent scalar utility values derived from their respective raw resource vectors (e.g., via L2-norm). The optimization goal is $d^* = \arg\max_{d \in \mathcal{D}_t} U(d, t)$. Based on this formulation, the optimization goal is $d^* = \arg\max_{d \in \mathcal{D}_t} U(d, t)$. To balance real-time performance with global optimality, we employ a *Greedy-Tabu Search* strategy (Algorithm 1), which initializes with a greedy candidate and iteratively explores a neighborhood $\mathcal{N}(d_{curr})$ while excluding nodes in the tabu list $\mathcal{L}_{tabu}$ to identify potential improvements.

D. Adaptation Optimization Component

Static configurations often lead to QoS degradation in dynamic environments. We employ a bidirectional adaptation mechanism to tune configurable parameters (e.g., sampling rate, batch size). Based on historical execution metadata, we model the computational resource consumption $\mathbf{C}(x)$ for a parameter x, and express the processing latency $L(x)$ as:

$$L(x) = \mathcal{F}(x, \mathbf{C}(x), \mathbf{r}_{curr}), \tag{8}$$

where $\mathcal{F}$ is the mapping derived from historical logs and $\mathbf{r}_{curr}$ denotes current available resources.

To guarantee service reliability, we solve a constrained optimization problem to find the optimal configuration x^* that maximizes application utility $\mathcal{U}_{app}(x)$ while satisfying the maximum tolerable latency L_{max} and resource limits:

$$\begin{aligned} x^* = \arg\max_{x} \quad & \mathcal{U}_{app}(x) \\ \text{s.t.} \quad & L(x) \leq L_{max}, \\ & \mathbf{C}(x) \leq \mathbf{r}_{curr}. \end{aligned} \tag{9}$$

By solving the above optimization problem, the system responds to resource fluctuations in real-time without violating QoS requirements.

Algorithm 1 Greedy-Tabu Search for Node Matching

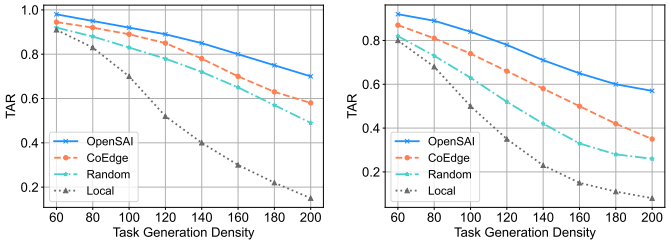
Require: Device set $\mathcal{D}_t$, search iterations I_{max}, tabu tenure τ, threshold θ
Ensure: Optimal allocation d^*

```
Calculate utility vector U = {U(d_i, t) | ∀d_i ∈ D_t} via Eq. (7)
Initialize d_curr ← arg max(U), d* ← d_curr
Initialize Tabu list L_tabu ← ∅
for k ← 1 to I_max do
    Generate neighborhood N = {d_j ∈ D_t | d_j ≠ d_curr, d_j ∉ L_tabu}
    if N = ∅ then
        break
    end if
    Select best neighbor d_next ← arg max_{d∈N} U(d, t)
    Update Tabu list: L_tabu ← L_tabu ∪ {d_curr}; remove oldest if |L_tabu| > τ
    d_curr ← d_next
    if U(d_curr, t) > U(d*, t) then
        d* ← d_curr
    end if
    if U(d*, t) ≥ θ then
        return d*
    end if
end for
if U(d*, t) < θ then
    return Local Execution
else
    return d*
end if
```

V. Experiments

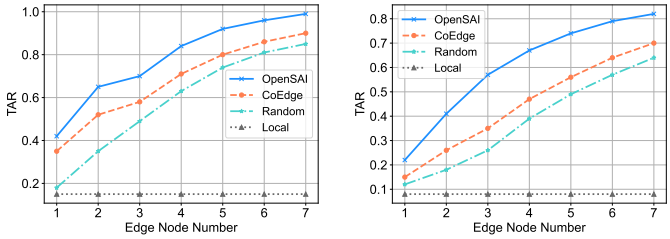
A. Simulation Settings

Testbed and Workloads. Our testbed comprises one master and three edge nodes, each a laptop equipped with an Intel Core i7-12700 CPU (14 cores, 16 GB RAM). We simulate a



(a) π Computation (b) Cooperative Perception

Fig. 3: Impact of task generation density on Task Acceptance Rate (TAR) for both workloads. (a) shows the results for π computation, and (b) shows the results for multi-agent cooperative perception.



(a) π Computation (b) Cooperative Perception

Fig. 4: Impact of the number of edge nodes on TAR for both workloads. (a) shows the results for π computation, and (b) shows the results for multi-agent cooperative perception.

dynamic topology where each edge node has a 10% probability of becoming unavailable per 100 ms scheduling cycle. The evaluation uses two representative workloads: π computation and multi-agent cooperative perception. The π computation task estimates π from 10^6 Monte Carlo samples; subtasks require 0.4∼0.5 s, aggregation 0.1 s, and the task deadline is 1.9 s. The cooperative perception task consists of parallel perception subtasks (0.1∼0.2 s each) and a final aggregation step (0.1∼0.2 s), with an overall deadline of 0.8 s. Faster R-CNN [23] is adopted as the object detector.

Parameter Settings. We set the heartbeat interval T_{heart} and new node discovery interval T_{new} to 100 ms, with a heartbeat timeout threshold of $N_{\text{heart}} = 5$. For the greedy-tabu scheduling algorithm (Algorithm 1), the number of search iterations is 10, the tabu list length is 2, and the selection threshold is $\theta = 0.7$. The node scoring weights in Eq. (7) are configured such that the network factor dominates with $\lambda = 0.5$.

Baselines and Metrics. We compare OpenSAI against three baselines: (i) Local, which executes all tasks on the master node; (ii) Random, which offloads entire, undecomposed tasks to a single random edge node; and (iii) CoEdge [24], a centralized dispatcher that considers network and computational resources. Evaluation is based on two metrics: (i) Task Acceptance Rate (TAR), the fraction of tasks completed before their deadlines ($N_{\text{finish}}/N_{\text{task}}$), and (ii) Weighted Average Completion Time (WACT), which penalizes failed tasks by using their deadline T_i^{dl} as their completion time. WACT is calculated as:

$$\text{WACT} = \frac{1}{N_{\text{task}}} \sum_{i=1}^{N_{\text{task}}} \left(\delta_i t_i + (1 - \delta_i) T_i^{dl} \right) \tag{10}$$

where δ_i is an indicator variable that equals 1 if task i is successful and 0 otherwise. All WACT results are normalized by the WACT of OpenSAI for easier comparison.

B. Quantitative Evaluation

We evaluate the performance of OpenSAI by systematically varying two key parameters: task generation density and the number of edge nodes. The former, defined as the number of tasks generated per minute, is used to simulate varying system loads. The latter represents the scale of available distributed resources for task offloading.

TABLE I: Impact of task generation density (Den.) on the Normalized Weighted Average Completion Time (WACT). All values are normalized by OpenSAI's WACT.

Method \ Den.	60	80	100	120	140	160	180	200
π Computation								
OpenSAI (Ours)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
CoEdge [24]	1.02	1.12	1.20	1.26	1.29	1.31	1.31	1.32
Random	1.05	1.10	1.22	1.33	1.40	1.44	1.46	1.47
Local	1.07	1.15	1.25	1.40	1.50	1.58	1.64	1.68
Multi-agent Cooperative Perception								
OpenSAI (Ours)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
CoEdge [24]	1.05	1.14	1.23	1.30	1.36	1.41	1.44	1.45
Random	1.11	1.19	1.28	1.36	1.45	1.53	1.58	1.60
Local	1.12	1.22	1.38	1.49	1.58	1.65	1.70	1.72

1) Impact of Task Generation Density: We evaluate the impact of task generation density on system performance, with other parameters as specified in Section V-A. As illustrated in Figure 3 and Table I, OpenSAI consistently outperforms all baselines across both workloads. Specifically, OpenSAI maintains significantly higher TAR results and achieves the lowest WACT metrics, demonstrating superior efficiency and reliability under increasing system load. Such performance gain is primarily attributed to two core mechanisms. First, the Computation Distribution Component performs fine-grained task partitioning, enabling high parallelism that sustains a robust TAR even at high densities by minimizing queuing delays. Second, the Cluster Management Component employs resource-aware scheduling to assign subtasks to capable nodes, effectively preventing bottlenecks and ensuring a low WACT. For the π computation task, characterized by highly parallel and independent subtasks, our system achieves near-optimal partitioning, resulting in minimal performance degradation as task density increases (Figure 3a). Conversely, while OpenSAI still leads, the TAR for cooperative perception shows a slightly more pronounced decline at higher densities (Figure 3b). We attribute this to the task's inherent data dependencies and the communication overhead associated with the final NMS fusion, which can become a centralized bottleneck. Nevertheless, OpenSAI's Adaptation Optimization Component mitigates this effect more effectively than baselines by dynam-

TABLE II: Impact of the number of edge nodes (Num.) on WACT. All values are normalized by OpenSAI's WACT.

Method \ Num.	1	2	3	4	5	6	7
π Computation							
OpenSAI (Ours)	1.00	1.00	1.00	1.00	1.00	1.00	1.00
CoEdge [24]	1.55	1.44	1.32	1.24	1.17	1.11	1.06
Random	1.62	1.58	1.47	1.37	1.28	1.19	1.11
Local	1.68	1.68	1.68	1.68	1.68	1.68	1.68
Multi-agent Cooperative Perception							
OpenSAI (Ours)	1.00	1.00	1.00	1.00	1.00	1.00	1.00
CoEdge [24]	1.62	1.53	1.45	1.37	1.28	1.19	1.13
Random	1.69	1.65	1.60	1.52	1.43	1.31	1.18
Local	1.72	1.72	1.72	1.72	1.72	1.72	1.72

ically adjusting task parameters.

2) Impact of Edge Node Number: To assess scalability, we evaluate system performance by varying the number of edge nodes from 1 to 7 under a constant, high-density workload (200 tasks/min). The experimental results are shown in Figure 4 and Table II. As Figures 4a and 4b shows, the TAR of OpenSAI exhibits a clear upward trend as the number of edge nodes increases, significantly outperforming the baselines. Table II shows that OpenSAI maintains the lowest normalized WACT across both tasks. In contrast, although CoEdge and Random achieve some improvements in WACT with an increasing number of nodes, their overall performance still lags far behind that of OpenSAI. Such scalability is directly attributed to two synergistic components. The Cluster Management Component dynamically discovers and integrates new nodes, enabling its scheduling algorithm to effectively distribute the load across a larger resource pool. Concurrently, the Adaptation Optimization Component fine-tunes task parameters in real-time to optimize resource utilization across the expanded cluster. Collectively, these mechanisms enable OpenSAI to efficiently harness additional resources, confirming its robust scalability and outperforming baseline approaches in dynamic edge environments.

VI. Conclusion

In this paper, we propose OpenSAI, a distributed cooperative computing platform designed for edge intelligence. OpenSAI adopts a three-layer architecture, centered around a core layer that integrates three key components, enabling fine-grained task parallelization, resource-aware node coordination, and dynamic parameter tuning based on runtime conditions. To validate its performance, we deploy OpenSAI in dynamic and heterogeneous edge environments using representative workloads. Experimental results demonstrate that OpenSAI significantly improves task success rates and reduces completion time, showcasing its enhanced efficiency, adaptability, and system stability.

References

[1] J. Liu, Y. Du, K. Yang, J. Wu, Y. Wang, X. Hu, Z. Wang, Y. Liu, P. Sun, A. Boukerche, and V. C. Leung, "Edge-cloud collaborative computing on distributed intelligence and model optimization: A survey," *IEEE Communications Surveys & Tutorials*, pp. 1–1, 2026.

[2] K. Yang, P. Sun, J. Lin, A. Boukerche, and L. Song, "A novel distributed task scheduling framework for supporting vehicular edge intelligence," in *ICDCS*, 2022, pp. 972–982.

[3] J. Liu, Y. Liu, J. Lin, J. Li, L. Cao, P. Sun, B. Hu, L. Song, A. Boukerche, and V. C. Leung, "Networking systems for video anomaly detection: A tutorial and survey," *ACM Computing Surveys*, vol. 57, no. 10, pp. 270:1–270:37, Apr. 2025.

[4] Z. Hong, W. Chen, H. Huang, S. Guo, and Z. Zheng, "Multi-hop cooperative computation offloading for industrial iot–edge–cloud computing environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 12, pp. 2759–2774, 2019.

[5] M. Hu, Z. Luo, A. Pasdar, Y. C. Lee, Y. Zhou, and D. Wu, "Edge-based video analytics: A survey," *arXiv preprint arXiv:2303.14329*, 2023.

[6] S. Baidya, Y.-J. Ku, H. Zhao, J. Zhao, and S. Dey, "Vehicular and edge computing for emerging connected and autonomous vehicle applications," in *DAC*. IEEE, 2020, pp. 1–6.

[7] J. Liu, L. Gong, J. Guo, J. Wu, L. Sun, Y. Bi, K. Patwari, B. Chen, L. Zhang, W. Zhou, Y. Liu, X. Zhu, C.-N. Chuah, and B. Rajaratnam, "Multimodal large language models in medicine and nursing: A survey," *techrxiv.175623882.21520632/v1*, 2025.

[8] D. Hu, W. Ji, and Z. Wang, "Multi-stream adaptive offloading of joint compressed video streams, feature streams, and semantic streams in edge computing systems," in *IEEE ICME*, 2023, pp. 996–1001.

[9] V. Balasubramanian, M. Wang, M. Reisslein, and C. Xu, "Edge-boost: Enhancing multimedia delivery with mobile edge caching in 5g-d2d networks," in *IEEE ICME*, 2019, pp. 1684–1689.

[10] A. Rocha Neto, T. P. Silva, T. Batista, F. C. Delicato, P. F. Pires, and F. Lopes, "Leveraging edge intelligence for video analytics in smart city applications," *Information*, vol. 12, no. 1, p. 14, 2020.

[11] J. Wu, J. Liu, Y. Liu, L. Wang, Z. Wang, W. Chen, Z. Tian, R. Yu, and V. C. M. Leung, "A survey on cloud-edge-terminal collaborative intelligence in aiot networks," *arXiv.2508.18803*, Aug. 2025.

[12] Y. Liu, J. Liu, C. Li, R. Xi, W. Li, L. Cao, J. Wang, L. T. Yang, J. Yuan, and W. Zhou, "Anomaly detection and generation with diffusion models: A survey," *arXiv:2506.09368*, Jun. 2025.

[13] J. Liu, P. Xie, W. Li, B. Tang, and J. Liu, "Dynamic task offloading scheme for edge computing via meta-reinforcement learning." *Computers, Materials & Continua*, vol. 82, no. 2, 2025.

[14] J. Liu, Z. Ma, Z. Wang, Y. Liu, Z. Wang, P. Sun, L. Song, B. Hu, and V. C. M. Leung, "A survey on diffusion models for anomaly detection," *arXiv:2501.11430*, Jan. 2025.

[15] Y. Gao, Z. Hou, C. Yang, Z. Li, H. Yu, and X. Li, "The prospect of enhancing large-scale heterogeneous federated learning with foundation models," in *IEEE ICME*, 2024, pp. 1–6.

[16] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH*, vol. 45, pp. 615–629, 2017.

[17] Z. Zhao, K. Wang, N. Ling, and G. Xing, "Edgeml: An automl framework for real-time deep learning on the edge," in *IoTDI*, 2021, pp. 133–144.

[18] Y. Liu, H. Li, X. Vasilakos, R. Hussain, and D. Simeonidou, "Cooperative task offloading through asynchronous deep reinforcement learning in mobile edge computing for future networks," *arXiv preprint arXiv:2504.17526*, 2025.

[19] A. Uddin, A. H. Sakr, and N. Zhang, "Task offloading in vehicular edge computing using deep reinforcement learning: A survey," *arXiv preprint arXiv:2502.06963*, 2025.

[20] J. Liu, J. Ren, Y. Zhang, X. Peng, Y. Zhang, and Y. Yang, "Efficient Dependent Task Offloading for Multiple Applications in MEC-Cloud System," *IEEE Trans. Mob. Comput.*, pp. 1–1, 2021.

[21] J. Meng, H. Tan, X.-Y. Li, Z. Han, and B. Li, "Online deadline-aware task dispatching and scheduling in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 6, pp. 1270–1286, 2019.

[22] Z. Ning, K. Zhang, X. Wang, L. Guo, X. Hu, J. Huang, B. Hu, and R. Y. Kwok, "Intelligent edge computing in internet of vehicles: A joint computation offloading and caching solution," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2212–2225, 2020.

[23] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *NeurIPS*, vol. 28, 2015.

[24] Z. Jiang, N. Ling, X. Huang, S. Shi, C. Wu, X. Zhao, Z. Yan, and G. Xing, "Coedge: A cooperative edge system for distributed real-time deep learning tasks," in *IPSN*, 2023, pp. 53–66.